

Práctica 2: Razonamiento con Restricciones

Autor: Jesús Muñoz Velasco

1. Problema de la fórmula de una bebida isotónica

a) A partir de la implementación se ha rellenado la siguiente tabla:

Volumen total buscado (en ml)	Primera solución encontrada	Número total de soluciones	<i>Runtime</i> (en ms) para calcular todas las posibles soluciones
200	Agua base: 0 ml Jarabe A: 0 ml Jarabe B: 120 ml Concentrado salino: 30 ml Extracto energetico: 40 ml Aroma citrico: 10 ml Coste total = 23.0 centimos Azucar total = 16.2 g Sodio total = 82.5 mg Cafeina total = 12.0 mg	6	198
1000	Agua base: 0 ml Jarabe A: 0 ml Jarabe B: 620 ml Concentrado salino: 150 ml Extracto energetico: 200 ml Aroma citrico: 30 ml Coste total = 115.8 centimos Azucar total = 83.0 g Sodio total = 415.5 mg Cafeina total = 60.0 mg	398	317
5000	Agua base: 0 ml Jarabe A: 0 ml Jarabe B: 3140 ml Concentrado salino: 710 ml Extracto energetico: 1000 ml Aroma citrico: 150 ml Coste total = 579.8 centimos Azucar total = 419.8 g Sodio total = 2013.5 mg Cafeina total = 300.0 mg	41031	8998
10000	Agua base: 0 ml Jarabe A: 0 ml Jarabe B: 6290 ml Concentrado salino: 1410 ml Extracto energetico: 2000 ml Aroma citrico: 300 ml Coste total = 1159.8 centimos Azucar total = 840.8 g Sodio total = 4011.0 mg Cafeina total = 600.0 mg	320957	40511

Tabla 1: Resultados de ejecución del Ejercicio 1a

b) Al minimizar el coste total obtenemos el siguiente resultado:

Volumen total buscado (en ml)	Solución óptima	Runtime (en ms)
200	Agua base: 0 ml Jarabe A: 0 ml Jarabe B: 110 ml Concentrado salino: 30 ml Extracto energetico: 40 ml Aroma citrico: 20 ml Coste total = 22.6 centimos Azucar total = 15.2 g Sodio total = 81.0 mg Cafeina total = 12.0 mg	118
1000	Agua base: 0 ml Jarabe A: 0 ml Jarabe B: 500 ml Concentrado salino: 160 ml Extracto energetico: 200 ml Aroma citrico: 140 ml Coste total = 111.2 centimos Azucar total = 70.8 g Sodio total = 415.0 mg Cafeina total = 60.0 mg	108
5000	Agua base: 0 ml Jarabe A: 0 ml Jarabe B: 2460 ml Concentrado salino: 770 ml Extracto energetico: 1000 ml Aroma citrico: 770 ml Coste total = 553.8 centimos Azucar total = 350.6 g Sodio total = 2016.5 mg Cafeina total = 300.0 mg	311
10000	Agua base: 0 ml Jarabe A: 0 ml Jarabe B: 4910 ml Concentrado salino: 1530 ml Extracto energetico: 2000 ml Aroma citrico: 1560 ml Coste total = 1107.0 centimos Azucar total = 700.4 g Sodio total = 4014.0 mg Cafeina total = 600.0 mg	1609

Tabla 2: Resultados de ejecución del Ejercicio 1b

c) Respondemos ahora a las preguntas planteadas:

- i) Si introducimos los tiempos encontrados en una gráfica y calculamos su curva de regresión obtenemos el siguiente resultado:

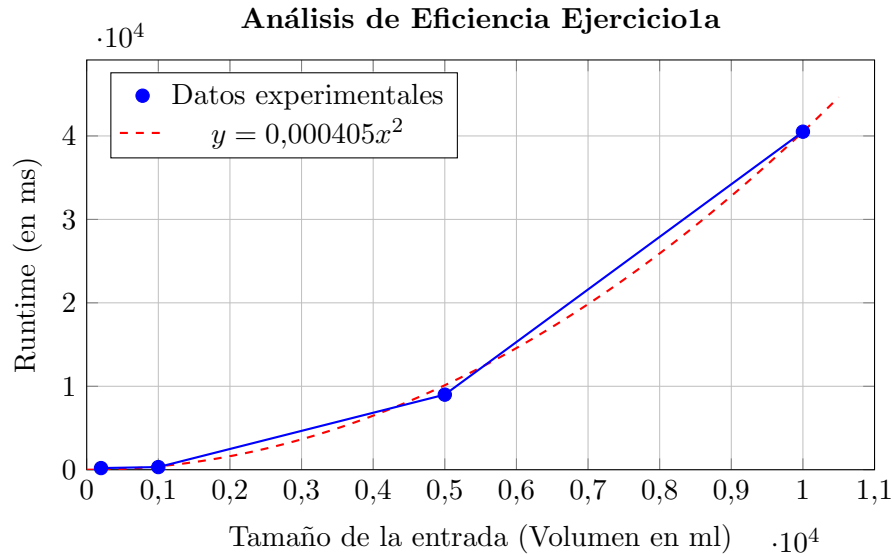


Figura 1: Representación de tiempos del ejercicio 1a

Por tanto parece que escala de forma cuadrática (eficiencia del orden $O(n^2)$). Por tanto, a priori, para $volumen_total_buscado_ml = 10^6$ (un millón) obtendríamos un tiempo aproximado de

$$t \approx 0,000405 \cdot (10^6)^2 = 405000000ms = 405000s = 112,5h = 4,6875 \text{ dias}$$

Por lo que tardaría más de 4 días de cómputo para calcular la solución (algo muy poco práctico).

- ii) A priori, para calcular la solución óptima (apartado b) necesitaríamos calcular todas las soluciones y después elegir la mejor de todas, por lo que el tiempo de ejecución debería ser prácticamente el mismo (ligeramente mayor en el caso de optimizar por el tema de las comparaciones). Sin embargo, observamos que tarda mucho menor en el caso de la optimización. Si comparamos los tiempos de satisfacción (Ejercicio 1a) y optimización (Ejercicio 1b) gráficamente obtenemos

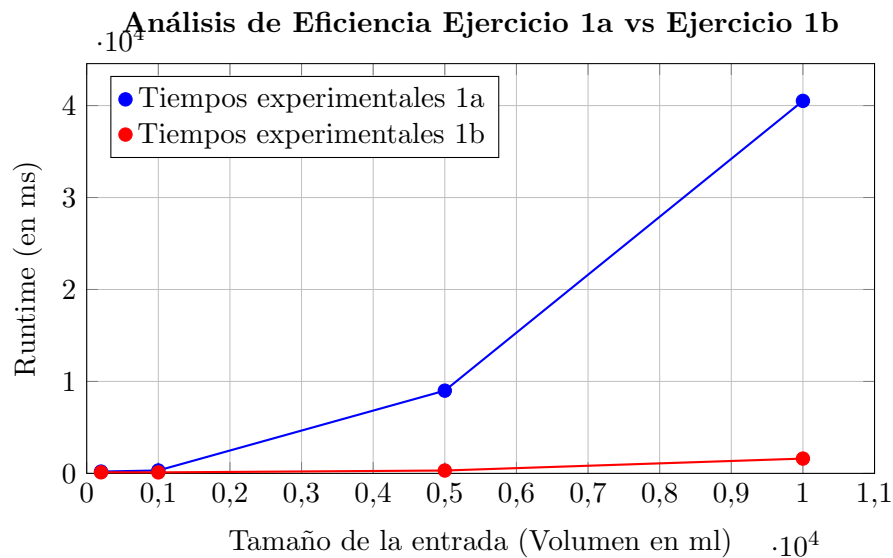


Figura 2: Comparación de tiempos entre satisfacción y optimización

Donde se ve mucho más claramente la diferencia. Por tanto vemos que la intuición no se cumple. Esto se debe a que MiniZinc no aplica fuerza bruta para resolver los ejercicios de optimización, sino que aplica algoritmos más eficientes (como *backtracking*). Si investigamos un poco descubrimos que el solver GECODE aplica varias estrategias para hacer esta búsqueda mucho más eficiente como la propagación de restricciones, la ramificación con heurísticas (como *First-Fail*) y el copiado híbrido para la marcha atrás (*backtracking*).

iii) Si ejecutamos el ejercicio que propone el enunciado llegamos a soluciones como

```
1 monedas: [1.469999999999998, 1.47, 1.47, 1.47, 1.47, 1.47, 0.0881999999999997,
           0.0, 0.0]
   total monedas: 8.908199999999979
```

Si añadimos una variable para ver qué obtenemos en la suma como la siguiente:

```
1 var float: valor_suma_monedas = sum(i in 1..n)(monedas[i]*valor[i]);
  % ...
  output [
    "monedas: " ++ show(monedas) ++ "\n" ++
5    "valor suma monedas: \"(valor_suma_monedas)\" \n" ++
    "total monedas: " ++ show(nmonedas)
  ];
```

obtenemos soluciones como

```
1 monedas: [1.469999999999998, 1.47, 1.47, 1.47, 1.47, 1.47, 0.0881999999999997, 0.0, 0.0]
   valor suma monedas: 1.469999999999999
   total monedas: 8.908199999999979
```

donde vemos que la suma no es 1,47 como se esperaba. Esto se debe principalmente a la representación flotante que comete errores en la aproximación (un flotante se representa internamente en binario, lo que puede inducir este tipo de errores, sobre todo a la hora de comparar).

Una opción para solucionar esto es aplicar la estrategia aplicada al ejercicio, es decir, utilizar representación entera pero multiplicada por un cierto valor (que se dividirá más adelante). Una posible solución aplicando esta representación sería la siguiente:

```
1 include "globals.mzn";
  int: importe_x100 = 147;
  % numero de monedas diferentes
  int: n = 9;
5  % valor de las monedas
  array[1..n] of int: valor_x100 = [1, 3, 5, 10, 25, 50, 100, 200, 500];
  % array para la cantidad de monedas de cada valor
  array[1..n] of var 0..importe_x100: monedas_x100;
  % numero de monedas usadas
10 var int: nmonedas_x100 = sum(i in 1..n) (monedas_x100[i]);
  % Valor de las monedas
  var int: valor_suma_monedas_x10000 = sum(i in 1..n)(monedas_x100[i]*valor_x100[i]);
  % La suma debe ser la cantidad buscada
  constraint valor_suma_monedas_x10000 == importe_x100 * 100;
15 solve satisfy;
  output [
    "monedas: [" ++
    [
20     "\"(monedas_x100[i]/100), \" | i in 1..n
    ] ++
    ["]\n" ++
    "valor suma monedas: \"(valor_suma_monedas_x10000/10000)\" \n" ++
    "total monedas: " ++ show(nmonedas_x100/100)
  ];
```

Con este cambio de representación conseguimos que todas las soluciones tengan la suma exacta (todas tienen valor `valor suma monedas: 1.47`).

2. Puzzle lógico

En este ejercicio se ha obtenido una única solución:

```
1 Alba: "A* cuántico", B1, 16:00
Bruno: "Poda alfa-beta aplicada al encaje de bolillos", B2, 15:00
Carla: "Demostración matemática de que RTA* es un algoritmo absurdo", A1, 18:00
Diego: "Modelado de la incertidumbre a la hora de aprobar TSI sin estudiar nada", A2, 17:00
5 Elena: "Planificación automática jerárquica híbrida dedicada a la yuxtaposición de los
mundos paralelos", C1, 19:00
Pablo: "Brazo robótico para jugar a la pocha", C2, 20:00
```

3. Problema de la mochila multiobjetivo con optimización lexicográfica

En este ejercicio se ha obtenido una única solución:

```
1 Objetos seleccionados:
- Camara (peso=2, coste=6, utilidad=9)
- Ordenador (peso=4, coste=10, utilidad=14)
- Bateria_extra (peso=2, coste=4, utilidad=7)
5 - Botiquin (peso=3, coste=5, utilidad=8)
- Radio (peso=4, coste=7, utilidad=11)

Utilidad total = 49
Peso total = 15
10 Coste total = 32
```

4. Problema del cuadrado mágico

Tras ejecutar el código proporcionado se han obtenido los siguientes resultados:

N	Núm Valores Distintos	Suma Común	Suma Total Minimizada	Runtime (ms)
3	5	15	45	201
4	8	24	96	213
5	13	37	185	632000
6	-	-	-	∞
7	-	-	-	∞

Tabla 3: Resultados de ejecución del Ejercicio 4

Si analizamos los tiempos obtenidos en una gráfica obtenemos lo siguiente:

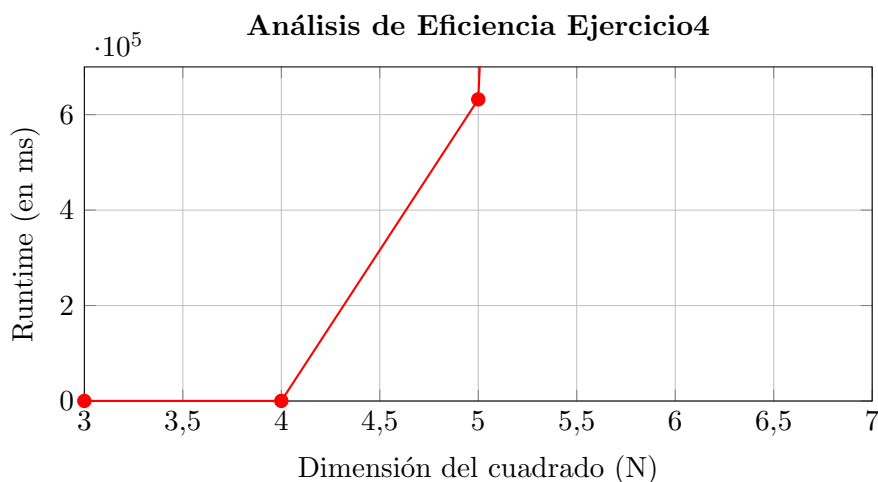


Figura 3: Representación de tiempos del ejercicio 4

Es fácil ver que el problema no es escalable, al menos con la configuración de mi código (probablemente admita más optimizaciones, incluso ajustando los dominios haciendo suposiciones, pero no concordarían con las condiciones del enunciado). El algoritmo parece ser de orden exponencial ($O(2^n)$) siendo muy malo para tamaños grandes (a partir de 5 ya es poco práctico). Si calculamos el crecimiento de posibilidades de una ejecución a la siguiente tendríamos el siguiente planteamiento:

Sea N el tamaño del cuadrado a resolver y M el máximo del dominio (consideramos que empieza en 0 por facilitar los cálculos aunque en realidad empezaría en 1). Tendríamos M valores posibles en cada casilla y tendríamos N^2 casillas. Esto hace que el número de cuadrados posibles a considerar sea M^{N^2} . Si consideramos que el algoritmo genera cada cuadrado y comprueba si se verifican las condiciones (en realidad no es así porque no utiliza fuerza bruta, pero sería el peor de los casos), estaríamos hablando de que el algoritmo tiene una eficiencia $O(2^{n^2})$ que es incluso peor que exponencial. Para el caso $M = 100$, $N = 4$ (que se resuelve rápido) tendríamos

$$M^{N^2} = 100^{4^2} = 100^{16} = 10^{32}$$

lo cual es inabarcable (este sería el peor de los casos recordemos, pero nos da una idea del crecimiento que tiene un caso con respecto al siguiente).

5. Problema de planificación de tareas

a) Si ejecutamos el script `Ejercicio5a.mzn` obtenemos la siguiente salida:

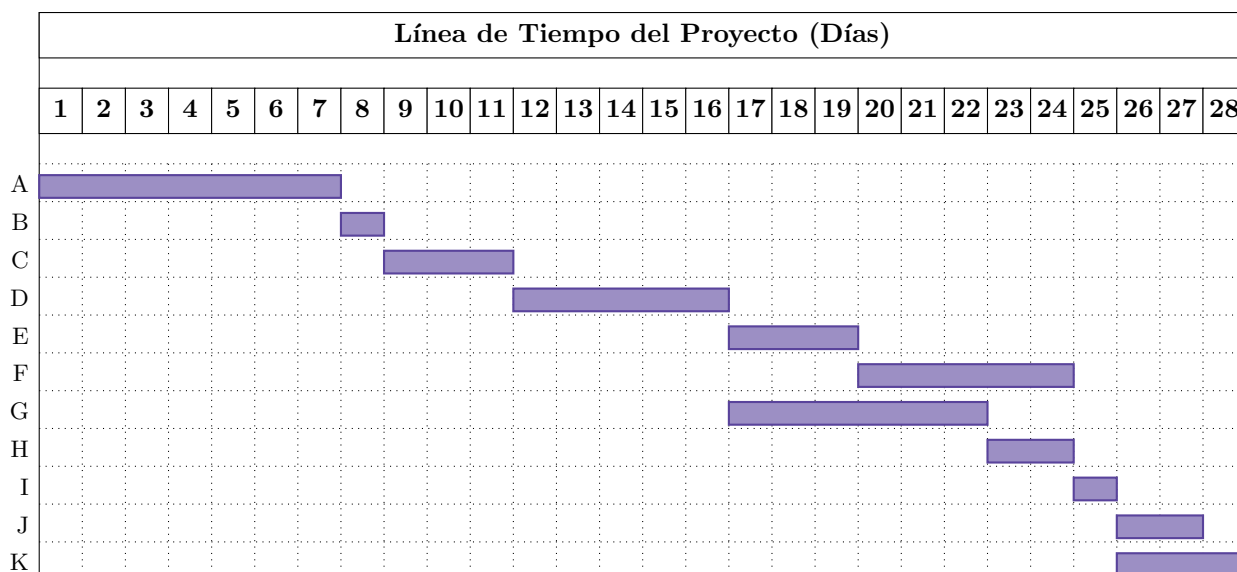
```

1  A - "Montar chasis" inicio dia 1 fin dia 7. Asignada a "Marty McFly" con apoyo de "Biff Tannen"
   B - "Instalar ruedas" inicio dia 8 fin dia 8. Asignada a "Marty McFly" con apoyo de "Nadie"
   C - "Cableado eléctrico" inicio dia 9 fin dia 11. Asignada a "Doc Brown" con apoyo de "Biff Tannen"
   D - "Motor de fusión nuclear" inicio dia 12 fin dia 16. Asignada a "Doc Brown" con apoyo de "Nadie"
5  E - "Tablero de control" inicio dia 17 fin dia 19. Asignada a "Doc Brown" con apoyo de "Biff Tannen"
   F - "Instalación y configuración del condensador de flujo (flux capacitor)" inicio dia 20 fin dia
   24. Asignada a "Doc Brown" con apoyo de "Nadie"
   G - "Ajuste aerodinámico" inicio dia 17 fin dia 22. Asignada a "Marty McFly" con apoyo de "George
      McFly"
   H - "Instalación de puertas de ala de gaviota" inicio dia 23 fin dia 24. Asignada a "Marty McFly"
      con apoyo de "Biff Tannen"
   I - "Panel de tiempo" inicio dia 25 fin dia 25. Asignada a "Marty McFly" con apoyo de "George McFly"
10  J - "Implementación y validación de algoritmos de búsqueda heurística para optimización de caminos"
      inicio dia 26 fin dia 27. Asignada a "Doc Brown" con apoyo de "Nadie"
   K - "Prueba de plutonio" inicio dia 26 fin dia 28. Asignada a "Marty McFly" con apoyo de "George
      McFly"

Duración total = 28 días

```

Y lo podemos traducir al siguiente diagrama de Gantt:



Id	Descripción de la tarea	Inicio	Fin	Responsable	Apoyo
A	Montar chasis	Día 1	Día 7	Marty McFly	Biff Tannen
B	Instalar ruedas	Día 8	Día 8	Marty McFly	—
C	Cableado eléctrico	Día 9	Día 11	Doc Brown	Biff Tannen
D	Motor de fusión nuclear	Día 12	Día 16	Doc Brown	—
E	Tablero de control	Día 17	Día 19	Doc Brown	Biff Tannen
F	Instalación y config. del condensador de flujo	Día 20	Día 24	Doc Brown	—
G	Ajuste aerodinámico	Día 17	Día 22	Marty McFly	George McFly
H	Instalación de puertas de ala de gaviota	Día 23	Día 24	Marty McFly	Biff Tannen
I	Panel de tiempo	Día 25	Día 25	Marty McFly	George McFly
J	Algoritmos de búsqueda heurística	Día 26	Día 27	Doc Brown	—
K	Prueba de plutonio	Día 26	Día 28	Marty McFly	George McFly

b) Si buscamos todas las soluciones que puedan plantearse en 28 días obtenemos 3 posibilidades, la ya estudiada en el apartado anterior y 2 más que son las siguientes:

1	A - "Montar chasis" inicio día 1 fin día 7. Asignada a "Marty McFly" con apoyo de "Biff Tannen"
	B - "Instalar ruedas" inicio día 8 fin día 8. Asignada a "Marty McFly" con apoyo de "Nadie"
	C - "Cableado eléctrico" inicio día 9 fin día 11. Asignada a "Doc Brown" con apoyo de "Biff Tannen"
	D - "Motor de fusión nuclear" inicio día 12 fin día 16. Asignada a "Doc Brown" con apoyo de "Nadie"
5	E - "Tablero de control" inicio día 17 fin día 19. Asignada a "Doc Brown" con apoyo de "Biff Tannen"
	F - "Instalación y configuración del condensador de flujo (flux capacitor)" inicio día 20 fin día 24. Asignada a "Doc Brown" con apoyo de "Nadie"
	G - "Ajuste aerodinámico" inicio día 17 fin día 22. Asignada a "Marty McFly" con apoyo de "George McFly"
	H - "Instalación de puertas de ala de gaviota" inicio día 23 fin día 24. Asignada a "Marty McFly" con apoyo de "Biff Tannen"
	I - "Panel de tiempo" inicio día 25 fin día 25. Asignada a "Marty McFly" con apoyo de "George McFly"
10	J - "Implementación y validación de algoritmos de búsqueda heurística para optimización de caminos" inicio día 27 fin día 28. Asignada a "Doc Brown" con apoyo de "Nadie"
	K - "Prueba de plutonio" inicio día 26 fin día 28. Asignada a "Marty McFly" con apoyo de "George McFly"
	Duración total = 28 días

15	A - "Montar chasis" inicio día 1 fin día 7. Asignada a "Marty McFly" con apoyo de "Biff Tannen"
	B - "Instalar ruedas" inicio día 8 fin día 8. Asignada a "Marty McFly" con apoyo de "Nadie"
	C - "Cableado eléctrico" inicio día 9 fin día 11. Asignada a "Doc Brown" con apoyo de "Biff Tannen"
	D - "Motor de fusión nuclear" inicio día 12 fin día 16. Asignada a "Doc Brown" con apoyo de "Nadie"
	E - "Tablero de control" inicio día 17 fin día 19. Asignada a "Doc Brown" con apoyo de "Biff Tannen"
20	F - "Instalación y configuración del condensador de flujo (flux capacitor)" inicio día 20 fin día 24. Asignada a "Doc Brown" con apoyo de "Nadie"
	G - "Ajuste aerodinámico" inicio día 17 fin día 22. Asignada a "Marty McFly" con apoyo de "George McFly"
	H - "Instalación de puertas de ala de gaviota" inicio día 23 fin día 24. Asignada a "Marty McFly" con apoyo de "Biff Tannen"
	I - "Panel de tiempo" inicio día 25 fin día 25. Asignada a "Marty McFly" con apoyo de "George McFly"
	J - "Implementación y validación de algoritmos de búsqueda heurística para optimización de caminos" inicio día 26 fin día 28. Asignada a "Marty McFly" con apoyo de "George McFly"
25	K - "Prueba de plutonio" inicio día 26 fin día 28. Asignada a "Doc Brown" con apoyo de "Nadie"
	Duración total = 28 días

Donde podemos ver que solo se producen cambios en las tareas J y K.

6. Problema de conformación de tribunales de defensa de TFG

Tras ejecutar el script Ejercicio6.mzn obtenemos 4 soluciones:

en la práctica anterior (las decisiones de diseño las he tomado de forma independiente a la IA).

Además, al final de cada ejercicio le entregaba la resolución (código MiniZinc), el resultado obtenido de la ejecución y el enunciado. A partir de esta información le pedía que comprobara si cumplía con todas las restricciones o si encontraba algún error lógico. En su mayoría decía que estaba todo bien o corregía algún fallo pequeño.